

课程笔记

Google Cloud 总结的 5 种 Agent Skill 设计模式——让 Agent 稳定输出的工程实践

慢学 AI (B 站) / 笔记整理

2026 年 4 月 23 日



视频作者/频道：慢学 AI

发布日期：2026 年

视频时长：7 分 18 秒

视频链接：<https://www.bilibili.com/video/BV1fMwyzkEPQ>

原文出处：Google Cloud Tech Blog — *Five agent skill design patterns every ADK developer should know*

目录

1 引言：为什么你的 Agent Skill 输出总是不稳定？	3
1.1 开篇痛点：Skill 搭好了，输出还是乱	3
1.2 根因诊断：问题不在格式，在内容设计	3
1.3 五种模式总览：对应 Agent 的五类失控点	4
1.4 本章小结	5
2 知识管控与结构固化：Trapper 与 Generator	6
2.1 Trapper 模式：按需注入知识，拒绝一次性塞满	6
2.1.1 痛点：System prompt 膨胀与知识污染	6
2.1.2 原理：Skill 做判断，References 按需加载	6
2.1.3 落地示例：改 React 页面 vs. 写 SQL 查询	7
2.1.4 本质总结	7
2.2 Generator 模式：用模板 + 风格指南锁定输出结构	8
2.2.1 痛点：输出结构每次都在“现场重新决定”	8
2.2.2 原理：模板管“做什么”，风格指南管“怎么做”	8
2.2.3 落地示例：API 文档的标准化生成	8
2.2.4 适用场景	10
2.3 本章小结	10
3 审查标准与信息补全：Reviewer 与 Inversion	10
3.1 Reviewer 模式：把“检查什么”和“怎么检查”硬拆开	11
3.1.1 痛点：审查任务为什么经常推倒重来	11
3.1.2 原理：流程不动，规则可换	11
3.1.3 落地示例：三级分类的审查报告	11
3.2 Inversion 模式：先问后做，信息补齐再开工	13
3.2.1 痛点：Agent 太爱猜	13
3.2.2 原理：把流程反过来	13
3.2.3 门控：信息没齐不准开工	13
3.2.4 落地示例：规划客服机器人	14
3.3 本章小结	15
4 步骤管控与校验：Pipeline 模式	16
4.1 Pipeline 模式：把 workflow 写进 Skill，每步有门控	16
4.1.1 从一句 prompt 到一份 workflow	16
4.1.2 门控：步骤之间的显式校验点	17
4.1.3 双重价值：顺序清晰 + Context 干净	18
4.1.4 适用场景	18
4.2 本章小结	19
5 模式选择与组合策略：如何选型、如何叠加	19
5.1 选择路径图：一张决策树搞定选型	19
5.2 模式叠加：1+1>2 的组合打法	20

5.3 总结与延伸 22

5.4 从”堆 prompt”到”选模式”. 22

5.5 延伸思考：模式之外 22

5.6 行动建议 23

1 引言：为什么你的 Agent Skill 输出总是不稳定？

1.1 开篇痛点：Skill 搭好了，输出还是乱

你是不是也遇到过这种情况？Skill（能力定义文件，通常以 `skill.md` 形式存在）已经写好了，System prompt（系统级提示词）也配齐了，但 Agent 调用 Skill 时输出仍然不稳定——一会儿格式乱，一会儿漏步骤，一会儿又自信地答偏。

“skill 搭好了，skill.md 也写了，但 agent 用 skill 输出还是不稳定，一会儿格式乱，一会儿漏步骤，一会又自信地答偏。”¹

这不是个例，而是当前 Agent 工程实践中的普遍痛点。很多开发者在搭建 Skill 时，把大量精力花在了格式规范上：文件怎么命名、字段怎么排列、指令用什么语气。然而当 Skill 真正投入运行时，却发现同样的输入在不同时间触发，产出的结果千差万别。我们把这种输出不稳定的典型表现归纳为三类：

- **格式漂移**：今天生成的报告有三级标题，明天变成了平铺列表；字段顺序时前时后，下游解析频频报错。
- **步骤遗漏**：多步骤任务中，模型“中途偷懒”，跳过关键环节，导致输出看似完整、实则残缺。
- **主题偏离（幻觉式自信）**：模型在信息不足的情况下，仍然高度自信地生成与用户需求无关的内容，甚至编造不存在的事实。

这些问题不仅影响用户体验，更直接拖慢工程迭代速度——因为每一次输出不稳定，都意味着额外的人工校验和反复修正。

1.2 根因诊断：问题不在格式，在内容设计

那么，问题的根源到底在哪里？

Google Cloud Tech 团队在博客文章 *Five agent skill design patterns every ADK developer should know* 中给出了一个明确的判断：问题很少出在格式上。²

“他们认为问题很少出在格式上……现在主流的 agent 工具已经逐渐统一了 skill.md 的格式规范，真正的问题出在内容设计上，也就是 skill 里面的逻辑究竟该怎么组织。”³

ADK 与 Skill 格式规范现状

ADK（Agent Development Kit）是 Google 推出的 Agent 开发框架，旨在为开发者提供标准化的 Agent 构建能力。目前，主流 Agent 工具（包括 ADK、Claude Code 等）的 `skill.md` 格式规范已趋于统一——文件结构、指令写法、元数据字段的约定基本达成共识。这意味着，“怎么写文件”已经不再是瓶颈，真正的挑战在于“文件里写什么逻辑”。

为了找到答案，Google 团队研究了 Anthropic 和 Google 内部的 Skill 构建实践，总结出五种设计模式。⁴ 他们指出，用对了这些模式，Skill 的输出才能稳定、有质量。⁵

¹视频画面时间区间：00:00:00–00:00:12。

²视频画面时间区间：00:00:12–00:00:23。

³视频画面时间区间：00:00:23–00:00:35。

⁴视频画面时间区间：00:00:35–00:00:42。

⁵视频画面时间区间：00:00:42–00:00:50。

核心观点

真正可靠的 Skill，靠的不是更复杂的指令堆砌，而是更合适的**内容结构**。格式规范已经统一，瓶颈在于 Skill 内部的逻辑组织方式——知识怎么加载、结构怎么固定、审查怎么做、信息怎么补全、步骤怎么管控。

1.3 五种模式总览：对应 Agent 的五类失控点

这五种模式对应的正是 Agent 最常见的五类失控场景，形成一张清晰的“问题 → 模式”映射表：⁶

表 1: Agent 失控点与五种 Skill 设计模式映射表

失控点	模式名	核心作用
知识什么时候加载	Trapper （知识加载模式）	按需注入知识，拒绝一次性塞满
输出结构不稳定	Generator （输出结构稳定模式）	用模板 + 风格指南固化输出
审查标准和流程混在一起	Reviewer （审查分离模式）	检查标准与检查流程分离
信息不全却急着开工	Inversion （信息补全模式）	先问后做，信息补齐再生成
复杂任务跳步骤	Pipeline （步骤管控模式）	把 workflow 写进 Skill，每步有门控

⁶视频画面时间区间：00:00:50–00:01:13。



图 1: 五类失控点与五种模式的映射关系。每一种模式都精准对应 Agent 在特定环节上的失控风险，从左到右形成“输入侧知识管控 → 输出侧结构固化 → 审查阶段标准分离 → 启动阶段信息补全 → 全流程步骤管控”的完整闭环。⁷

常见误区：把五种模式当成互斥选项

初学者容易误以为五种模式只能“五选一”。实际上，它们之间可以互相叠加组合——Trapper 几乎可以叠在任何模式上负责按需补知识；Generator 前面可以接 Inversion 做信息补全，结尾可以接 Reviewer 做质量审查；Pipeline 则可以包裹整个流程。模式选对了，组合用好了，Agent 的输出才能真正稳定。

接下来，我们将逐一深入讲解这五种模式。先看第一种 **Trapper**，它解决的是“知识什么时候加载”的问题。

1.4 本章小结

本章从 Agent Skill 输出不稳定的普遍痛点出发，剖析了问题的真正根源——不在格式规范，而在内容设计的逻辑组织方式。我们引入了 Google Cloud Tech 团队总结的五种 Skill 设计模式，建立了“失控点 → 模式”的映射框架。这五种模式构成了一个从输入到输出、从启动到收尾的完整工程方法论。下一章我们将从输入侧开始，深入讲解 **Trapper** 和 **Generator** 两种模式的具体原理与落地方法。

⁷ 视频画面时间区间：00:00:50–00:01:13。

2 知识管控与结构固化：Trapper 与 Generator

从本节开始，我们将逐一拆解五种设计模式。先看第一种——**Trapper（知识加载模式）**，它解决的是”知识什么时候加载”的问题；随后是第二种——**Generator（输出结构稳定模式）**，它解决的是”输出结构总是不稳定”的问题。两者分别从输入侧和输出侧对 Agent 的行为进行管控，是一对互补的基础模式。

2.1 Trapper 模式：按需注入知识，拒绝一次性塞满

2.1.1 痛点：System prompt 膨胀与知识污染

在实际开发中，一个常见的直觉是：既然 Agent 需要遵守规范，那就把全部规范提前写进 **System prompt（系统提示词）**。React 组件规范、FastAPI 接口约定、团队命名规则、Git 提交规范……一次性全塞进去，仿佛准备得越充分，Agent 表现就越可靠。

然而，这种做法带来了两个隐蔽却严重的问题：

System prompt 膨胀的代价

1. **浪费 Context**：大部分任务根本用不到全部规范，无关内容白白占用模型上下文窗口；
2. **引入干扰**：模型更容易被无关知识带偏，在不该考虑前端规范的任务里”回忆”起 React 的约束，反而降低输出质量。

视频中的原话一针见血：

”很多人会把某个技术栈的全部规范提前塞进 system prompt……问题是大部分任务根本用不到这些东西，你在浪费 context，模型还更容易被无关知识干扰。”（视频 01:13–01:39）

2.1.2 原理：Skill 做判断，References 按需加载

Trapper 模式的核心思路是”**按需注入（On-demand Injection）**”——不要把知识直接写进 skill，而是把知识放进 **References（参考资料）**，由 skill 本身只负责一个判断：当前任务是否进入了某个特定领域？

只有当判断”命中”时，对应的那一套知识才被加载进当前上下文。

Trapper 模式的核心机制

- **References**：外部知识文档，存放具体的技术规范、团队约定、命名规则等；
- **Skill（闸门）**：只负责判断任务领域，决定何时加载哪一份 references；
- **按需注入**：未命中的知识绝不进入当前 context，保持上下文干净。

这个机制可以用一个日常类比来理解：skill 像一道**总闸门（Gatekeeper）**，负责在合适的时候把某本”专家手册”接入上下文。图书馆里藏书万千，但读者桌面上只放当前需要的那一本——Trapper 做的就是这件事。

”Trapper 的做法是把这些内容放进 references，skill 只负责判断现在这个任务是否进入了 React 领域……只有判断命中才加载那一套知识。你可以把它理解成 skill 像一个总闸门，负责在合适的时候把某个专家手册接入上下文。”（视频 01:39-02:01）

2.1.3 落地示例：改 React 页面 vs. 写 SQL 查询

假设你的团队维护着一套完整的技能库，其中 references 目录下存放着多份规范文档：

- react-coding-standards.md —— React 组件开发规范
- sql-style-guide.md —— SQL 编写与优化规范
- copywriting-guidelines.md —— 产品文案风格指南

场景 A：你让 Agent 改一个 React 页面。skill 判断任务进入了”前端/React”领域，于是加载 react-coding-standards.md，Agent 在修改时遵循组件命名、Hooks 使用规则、样式隔离等约束。

场景 B：当前任务只是写一条 SQL 查询，或者改一段产品文案。skill 判断任务未命中 React 领域，那份前端规范**根本不出现**，context 中只保留与 SQL 或文案相关的知识。

”比如让 Agent 改一个 React 页面时，再加载 references 里的 React 规范；但如果当前任务只是写 SQL 查询，或者只是改一段产品文案，那份前端规范根本不出现。”（视频 02:01-02:16）



图 2: Trapper 模式工作示意: Skill 作为总闸门，按需将专家手册接入上下文。来源: 视频 01:13-02:16

2.1.4 本质总结

Trapper 的本质就是四个字：**按需注入知识**。它把”有什么知识”和”什么时候用知识”拆成了两个独立的问题——前者由 references 回答，后者由 skill 的闸门逻辑回答。这种分离不仅节省了 token，更重要的是让模型在每一次调用时都面对一个干净、聚焦的上下文。

Trapper 常见误区

闸门逻辑写得过于模糊是 Trapper 模式最常见的失败原因。如果 skill 判断“当前任务是否进入 React 领域”的规则只有一句“如果涉及前端就加载”，那么几乎所有任务都会触发误加载——写一条 SQL 查询也可能因为提到“前端展示”而被注入 React 规范。判断条件必须具体、可验证，否则 Trapper 会从“按需注入”退化为“随机注入”。

2.2 Generator 模式：用模板 + 风格指南锁定输出结构

2.2.1 痛点：输出结构每次都在“现场重新决定”

Trapper 解决了输入侧的知识污染问题，但 Agent 的输出侧还有一个同样棘手的难题：为什么同样让 Agent 写一份 API 文档，今天长这样，明天又长成另一种样子？

根源在于，模型每次都在现场重新决定这份输出该长什么样。没有外部约束时，模型会基于当次上下文的概率分布“自由发挥”——这次把参数表放在前面，下次把错误码放在前面；这次用正式语气，下次又偏口语化。这种结构漂移对工程协作来说是灾难性的。

“本质原因是模型每次都在现场重新决定这份输出该长什么样。Generator 的答案很直接：别让他现场发挥，把输出结构固定在模板里，把表达方式固定在风格指南里。”（视频 02:16–02:40）

2.2.2 原理：模板管“做什么”，风格指南管“怎么做”

Generator 模式的解决方案由两部分组成：

Generator 模式的双支柱

1. **Template (模板)**：回答 *what to produce*，定义输出必须包含的字段、顺序和层级关系；
2. **Style Guide (风格指南)**：回答 *how to write*，定义标题格式、术语统一、语气（正式/口语）、字段命名规范（camelCase / snake_case）等。

skill.md 的职责也随之明确：先补问缺失变量，再把收集到的内容填入模板，最后对照风格指南做一致性检查。模型从“创作者”变成了“填空执行者”，结构漂移自然消失。

“模板回答的是 *what to produce*，风格指南回答的是 *how to write*。skill.md 负责做的是先补问缺失变量，再把内容填进去。”（视频 02:40–02:51）

2.2.3 落地示例：API 文档的标准化生成

假设你需要 Agent 自动生成后端接口文档。Generator 模式下的 skill 可以这样组织：

Template 定义：

1. 接口说明（功能概述、调用场景）
2. 请求参数表（字段名 / 类型 / 必填 / 说明）
3. 返回值说明（成功响应结构 / 字段含义）
4. 错误码列表（码值 / 含义 / 触发条件）

5. 示例调用（cURL 或 Python 代码）

Style Guide 规定：

- 标题格式：二级标题用接口路径，如 `POST /api/v1/users`；
- 术语统一：“用户”一律用“user”，禁止混用“member”；
- 语气：技术文档保持正式，禁止口语化表达；
- 字段命名：JSON 字段统一使用 `snake_case`。

”比如你要让 Agent 产出 API 文档，模板规定必须有接口说明、参数表、返回值、错误码、示例调用；style guide 规定标题格式、术语统一、语气和字段命名。这样你今天写、明天写、不同人触发，出来的结构还是同一套。”（视频 02:51–03:17）



图 3: Generator 模式工作流：模型不再“现场发挥”，而是按模板填空、按风格指南校验。来源：视频 02:16–03:17

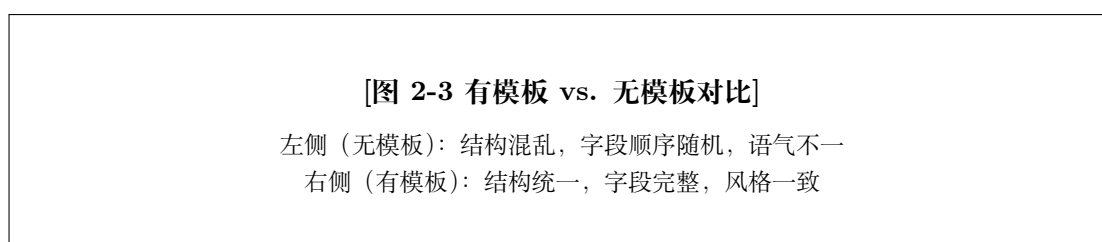


图 4: 同一 API 文档在“无模板”与“有模板”两种模式下的输出对比。来源：视频 02:16–03:17

2.2.4 适用场景

Generator 模式特别适合以下场景：

- **API 文档生成**：接口结构固定，团队需要长期一致的文档格式；
- **标准化报告**：日报、周报、月报等重复性文档，模板化后可大幅降低审阅成本；
- **Commit message 脚手架**：统一提交信息格式（类型、范围、描述、关联 issue）；
- 任何需要“今天写、明天写、不同人触发，结构还是同一套”的输出任务。

Generator 与 Trapper 的协同关系

Trapper 和 Generator 并非互斥，反而天然互补：Trapper 负责在输入侧按需加载知识（比如加载 API 文档规范），Generator 负责在输出侧用模板固化结构。一个管“进来什么”，一个管“出去什么样”，两者叠加即可实现“知识精准 + 结构稳定”的双重保障。

Generator 常见误区

模板过度僵化是 Generator 模式最常见的陷阱。当模板字段被设计得过于死板时，模型在边缘场景下会强行“填空”——比如模板要求“必须列出 5 个参数”，但某个接口实际上只有 2 个参数，模型就可能编造 3 个不存在的字段来完成模板。好的模板应当区分“必填项”和“条件项”，并允许在备注中说明“无此项时留空”。

2.3 本章小结

本章介绍了五种模式中的前两种，它们分别作用于 Agent 的输入端和输出端：

- **Trapper 模式**解决的是**输入侧的知识污染**问题。它把全部规范从 System prompt 中移出，存放到 References 中，由 Skill 充当总闸门，只在任务命中对应领域时才加载相关知识。核心收益是 context 干净、token 节省、干扰减少。
- **Generator 模式**解决的是**输出侧的结构漂移**问题。它通过 Template（定义 what to produce）和 Style Guide（定义 how to write）把输出结构预先固定，让模型只负责填空而非创作。核心收益是输出一致、可预期、可协作。

两者的关系可以概括为：**Trapper 管输入，Generator 管输出**。Trapper 确保 Agent“看对了书”，Generator 确保 Agent“写对了格式”。在实际工程中，这两种模式常常叠加使用——Trapper 按需加载 Generator 所需的模板和风格指南，共同构成 Agent 稳定输出的第一道防线。

接下来，我们将进入第三种模式——**Reviewer（审查分离模式）**，它解决的是审查任务经常要“推倒重来”的问题。

3 审查标准与信息补全：Reviewer 与 Inversion

第三种 **Reviewer（审查分离模式）**和第四种 **Inversion（信息补全模式）**，分别对应 Agent 在“收尾”和“启动”两个阶段的失控点。Reviewer 解决“已有内容怎么审”的问题，Inversion 解决“新任务怎么开始”的问题。两者一前一后，恰好覆盖了任务生命周期的两个关键节点。

3.1 Reviewer 模式：把”检查什么”和”怎么检查”硬拆开

3.1.1 痛点：审查任务为什么经常推倒重来

在工程实践中，让 Agent 做代码审查、文档审查或合规审查，是常见的需求。但许多团队很快会发现一个令人沮丧的现象：审查 skill 刚搭好，换个审查主题就要重写一遍。昨天还在检查 Python 的 PEP8 风格，今天想换成 OWASP 安全检查，后天又要做企业合规审查——每一次都像是从零开始。

视频中的原话精准地描述了这一痛点：

”很多团队把两件事写混了：一件事是检查什么，另一件事是怎么检查。Reviewer 模式要求你把这两件事硬拆开。”
——视频 03:25

问题的根因在于，开发者往往把”检查标准”（即审查的内容和规则）与”检查流程”（即审查的执行方式）混在同一个 skill.md 文件中。当审查主题变化时，不得不连流程一起重写，造成大量重复劳动。

3.1.2 原理：流程不动，规则可换

Reviewer 模式的核心思想是 **流程与规则解耦 (Process-Rule Decoupling)**。具体做法是将两者硬拆开：

- **检查什么**——放进 **Checklist (检查清单)**。Checklist 是可替换的规则集，定义了当前审查任务需要检查的具体条目。
- **怎么检查**——留在 **Skill.md** 中。Skill.md 定义的是稳定的审查流程框架，包括接收输入、逐条对照、标记级别、输出报告等步骤。

这种分离带来的直接好处是：**流程不动，规则可换**。同一套审查 skill，只需替换不同的 checklist，就能胜任完全不同的审查任务。

Reviewer 模式的核心价值

将”检查标准”与”检查流程”解耦，使审查引擎成为可复用的基础设施，而 checklist 成为可插拔的规则模块。

视频中的原话进一步阐释了这一价值：

”流程不动，规则可换。比如你今天做 Python 风格审查，明天换成 OWASP 安全检查，后天又换成合规规则，你不需要为每一类审查重写一遍 skill，只需要替换不同的 checklist。”
——视频 03:40

3.1.3 落地示例：三级分类的审查报告

一个典型的 Reviewer skill.md 中的审查流程可以设计如下：

1. **接收输入**：获取待审查的内容（代码、文档、配置等）。
2. **逐条对照 Checklist**：将输入内容与当前 checklist 中的每一条规则进行比对。
3. **标记级别**：对发现的问题按严重程度分类。

4. 输出审查报告：以结构化格式呈现审查结果。

其中，**输出层级标准化**是 Reviewer 模式的重要补充。审查结果不应是简单的一段文字，而应统一为三级分类：

- **Error (错误)**：必须修复，否则无法通过审查。
- **Warning (警告)**：建议修复，存在潜在风险。
- **Info (信息)**：仅作提醒，供开发者参考。

视频原话对此有明确说明：

”最后输出再统一成 Error、Warning、Info 这种层级，这样人一看就知道哪些必须修，哪些建议修，哪些只是提醒。”
——视频 03:56

可替换的 checklist 示例如下：

- **Checklist A**：Python PEP8 风格规范（缩进、命名、导入顺序等）。
- **Checklist B**：OWASP Top 10 安全漏洞检查（注入、身份验证、敏感数据暴露等）。
- **Checklist C**：企业合规规则（数据隐私、授权审批、日志留存等）。



图 5: Reviewer 模式架构图：底层为稳定的审查流程引擎，上层为可插拔的 checklist 模块。来源：视频 03:17-04:04

常见误区：把级别判断也写进 checklist

有些开发者会把”某条规则属于 Error 还是 Warning”的判断逻辑也放进 checklist。实际上，级别的判定标准应当属于 skill.md 中的流程定义（例如”任何涉及安全漏洞的问题都标记为 Error”），而 checklist 只负责列出”检查什么”。否则，换 checklist 时级别逻辑也会跟着乱。

3.2 Inversion 模式：先问后做，信息补齐再开工

3.2.1 痛点：Agent 太爱猜

第四种 Inversion 模式，是五种模式中最容易让人产生实感的一种。原因很简单：几乎所有 LLM 都有一个通病——**自信幻觉 (Overconfident Hallucination)**。拿到一个模糊需求，Agent 不会停下来追问，而是急着开始生成；信息不够，就自己“脑补”，而且补得非常自信。

视频中的描述极具画面感：

”这是很多 agent 最大的问题是太爱猜，一拿到一个模糊需求就急着开始生成，信息不够他就自己补，而且补得非常自信。”
——视频 04:11

这种“先猜后改”的模式，导致的结果往往是：生成了一大堆内容，用户看了才发现完全不是自己想要的，然后进入漫长的反复修改循环。问题的根因不在生成能力，而在启动阶段的信息缺失。

3.2.2 原理：把流程反过来

Inversion 模式的思路是把传统的“需求 → 生成 → 修改”流程，反转为“需求 → 澄清 → 确认 → 生成”流程。具体来说，Agent 不再直接生成内容，而是先发问，由用户回答，信息齐了才允许开工。

视频原话清晰地概括了这一反转：

”Inversion 的思路是把流程反过来……通常会分成三个阶段：第一阶段 discovery 先确认你到底要解决什么问题；第二阶段 constraints 把平台技术栈限制条件交付要求问完整；第三阶段 synthesis 信息齐了之后才允许开始生成。”
——视频 04:24

这三个阶段构成了 Inversion 模式的完整框架：

1. **Discovery (发现阶段)**：确认用户到底要解决什么问题。很多时候用户提出的“需求”只是表象，真正的业务问题需要进一步挖掘。
2. **Constraints (约束阶段)**：把平台、技术栈、限制条件、交付要求等关键信息问完整。这是防止 Agent“脑补”的核心防线。
3. **Synthesis (合成阶段)**：信息齐全之后，才允许开始生成方案或内容。

Inversion 模式的三阶段

Discovery → Constraints → Synthesis。前两个阶段的信息收集不到位，第三阶段绝不放行。

3.2.3 门控：信息没齐不准开工

Inversion 模式能否生效，关键在于一个显式的 **门控指令 (Gate)**。Skill.md 中必须用明确的规则规定：在 Discovery 和 Constraints 阶段的信息未收集完整之前，Agent 禁止进入 Synthesis 阶段。视频中的原话强调了这一机制的重要性：

”所以 Inversion 的关键是用门控指令明确规定：信息没齐不准开工。” ——视频 05:01

这个门控不是建议，而是强制规则。它应当被明确写入 skill.md 的指令中，例如：

[Gate Rule]

在以下信息全部确认之前，禁止输出任何方案或代码：

1. 服务对象（C端用户 / B端客户 / 内部员工）
2. 接入渠道（微信 / 网页 / App / 电话）
3. 是否需要对接工单系统
4. 是否有合规要求（数据隐私、行业监管）
5. 是否有人工接管机制

3.2.4 落地示例：规划客服机器人

假设用户给了一个模糊需求：“帮我规划一个客服机器人。”在传统模式下，Agent 可能会直接输出一份通用的客服机器人方案——但这份方案很可能与用户的实际场景相去甚远。

在 Inversion 模式下，Agent 会按三阶段逐步澄清：

• Discovery 阶段：

- 这个客服机器人的服务对象是谁？（C 端消费者、B 端企业客户、还是内部员工？）
- 主要解决哪类问题？（售前咨询、售后支持、技术支持、投诉处理？）

• Constraints 阶段：

- 接入渠道是什么？（微信公众号、网页插件、独立 App、电话语音？）
- 是否需要对接现有的工单系统或 CRM？
- 有没有行业合规要求？（如金融行业的数据隐私、医疗行业的信息保密）
- 是否需要人工接管机制？什么条件下转人工？
- 技术栈有无限制？（必须用 Python、必须用某家云厂商的 AI 服务？）

• Synthesis 阶段：

- 以上信息全部确认后，才开始输出完整的客服机器人架构方案。

视频中的原话以这个案例做了生动说明：

”比如你让 agent 规划一个客服机器人，他应该先问服务对象是谁，接入微信还是网页，要不要对接工单系统，有没有合规要求，有没有人工接管机制。这些没问清之前，禁止直接开始写方案。”
——视频 04:48



图 6: 传统流程与 Inversion 流程的对比。左侧为”先猜后改”的多轮循环，右侧为”先问后做”的一次性生成。来源：视频 04:04–05:07

为什么叫”Inversion”？

Inversion 的字面含义是”反转”。在软件工程中，这一命名呼应了**控制反转（Inversion of Control, IoC）**的设计思想——传统的控制流是”用户提需求，Agent 直接执行”；Inversion 模式将控制流反转为”Agent 主动收集信息，满足条件后才执行”。这种命名上的呼应并非巧合，它暗示了两种模式在哲学层面的共通性：把控制权从”执行方”交还给”条件方”。

常见误区：把”提问”本身当成输出

有些 skill 虽然写了”请先提问”，但 Agent 在提问的同时就开始输出方案草稿，导致用户误以为”它已经开始了”。门控规则必须明确禁止任何形式的”预生成”——哪怕是提纲、框架或思路摘要，在信息补齐之前都不应出现。否则用户会提前形成预期，反而干扰后续的澄清过程。

3.3 本章小结

Reviewer 模式和 Inversion 模式分别解决了 Agent 任务生命周期中”收尾”和”启动”两个关键节点的失控问题。

Reviewer 模式的核心贡献是**流程与规则解耦**。通过将”检查什么”（checklist）与”怎么检查”（skill.md 中的审查流程）硬拆开，团队可以用同一套审查引擎应对 Python 风格审查、OWASP 安全检查、企业合规审查等完全不同的场景。配合 Error / Warning / Info 三级输出分类，审查结果的可读性和可执行性大幅提升。

Inversion 模式的核心贡献是**信息补齐前置**。通过 Discovery → Constraints → Synthesis 的三阶段流程，配合显式的门控规则”信息没齐不准开工”，根治了 Agent”拿到模糊需求就急着生成”的自信幻觉问题。以客服机器人规划为例，只有在服务对象、接入渠道、工单系统、合规要求、人

工接管机制等信息全部确认后，Agent 才被允许输出方案。

两者形成了一种对称关系：Reviewer 确保”产出之后有标准地审”，Inversion 确保”开工之前有充分地问”。一前一后，共同压缩了 Agent 在”启动”和”收尾”两个阶段的失控空间。接下来，我们将进入第五种模式——Pipeline，它解决的是多步骤复杂任务中”中途跳步”的问题。

4 步骤管控与校验：Pipeline 模式

4.1 Pipeline 模式：把 workflow 写进 Skill，每步有门控

前面四种模式分别解决了知识加载、结构固化、审查分离和信息补全的问题。但在实际工程中，还有一类任务特别让人头疼——它们天然就是多步骤的强顺序流程，比如分析输入、生成中间产物、组装最终结果、最后还要验证。Agent 执行这类任务时，最大的风险不是某一步做错，而是**直接跳过某一步**，仿佛一个爱偷懒的实习生，看到流程复杂就自动省略中间环节。

Pipeline 模式的核心定义

Pipeline 模式 (Pipeline / Step Control Pattern) 将复杂任务拆分为强顺序的多步骤 workflow，并把每一步的输入、输出、校验条件和推进规则显式写进 `skill.md`。步骤之间设置**门控** (Gate / Checkpoint)，只有满足条件才能进入下一步，从根本上杜绝”中途偷懒”。

视频里对这类痛点的描述非常精准：

”有些任务本身就是一个强顺序流程，比如先分析输入，再生成中间产物，再组装最终结果，最后还要验证。如果你把这件事当成一句 prompt，模型很容易中途偷懒。”——视频原话，05:17-05:27

问题的根源在于：当你用一句长 prompt 描述整个流程时，模型需要同时理解”做什么”和”按什么顺序做”。context 越长，模型越容易”选择性执行”——它可能直接跳到看起来最相关的步骤，或者把中间步骤合并成一步草率完成。这不是模型”故意”犯错，而是**长指令中的顺序约束天然容易被稀释**。

4.1.1 从一句 prompt 到一份 workflow

Pipeline 的解决方案非常直接：不再把流程塞进一句 prompt，而是把 `skill.md` 本身写成一份 **工作流文档** (Workflow Document)。具体来说，你需要明确定义：

1. **步骤清单**：第一步做什么，第二步做什么，第三步做什么……
2. **推进条件**：什么时候能进入下一步？需要产出什么中间件？需要谁确认？
3. **门控规则** (Gate)：步骤之间的显式校验点，未通过则停留当前步骤，禁止推进。

”Pipeline 的做法是直接写 `skill.md` 成工作流，定义第一步做什么，第二步做什么，什么时候能进入下一步，全都写清楚，中间还要放 gate，也就是显式门控。”——视频原话，05:32-05:43

4.1.2 门控：步骤之间的显式校验点

门控 (Gate) 是 Pipeline 模式区别于普通“分步骤提示”的关键机制。它不是一句“请检查后再继续”的礼貌请求，而是一条硬性规则：当前步骤的输出必须满足特定条件，否则流程锁死在当前步骤，不得进入下一步。

一个典型的 Pipeline 示例是 **API 文档生成**：

1. **步骤 1：解析公开 API**——读取 OpenAPI 规范或源码中的接口定义，提取参数、返回值、错误码等信息。
2. **步骤 2：生成 doc string**——为每个接口生成标准化的文档字符串，描述功能、参数含义和返回值类型。
3. **步骤 3：确认后组装成完整文档**——将 doc string 组装为完整的 API 参考文档，包含目录、示例和错误码对照表。
4. **步骤 4：最后做质量检查**——检查文档完整性、术语一致性、链接可访问性等。

在这个流程中，门控可以设置在：

- 步骤 1 → 步骤 2 之间：确认所有 API 端点已解析完毕，无遗漏。
- 步骤 2 → 步骤 3 之间：确认 doc string 已通过人工或自动校验，参数描述无歧义。
- 步骤 3 → 步骤 4 之间：确认文档结构完整，所有接口均已覆盖。

”比如先解析公开 API，再生成 doc string，确认之后再组装成完整文档，最后做质量检查。”
——视频原话，05:43–05:49



图 7: Pipeline 模式以横向管道形式组织任务，阶段之间用菱形 Gate 节点做显式校验。来源：视频 05:32–05:49

4.1.3 双重价值：顺序清晰 + Context 干净

Pipeline 模式的价值不仅体现在”不跳步”上，还有一层容易被忽略的工程收益：**每一步只加载当前所需的 references 和 templates，context 始终保持干净。**

在传统的”单 prompt 模式”中，你把所有参考资料——API 规范、文档模板、风格指南、测试用例——一次性塞进 context。模型在执行步骤 1（解析 API）时，不得不同时”看到”步骤 4 才会用到的质量检查清单。这些信息虽然当前用不到，却占用了宝贵的 context 窗口，还可能引入干扰。

Pipeline 模式通过**分段加载**（Context Segmented Loading）解决了这个问题：

- 步骤 1 只加载 API 解析相关的 references（如 OpenAPI 规范、源码目录结构）。
- 步骤 2 只加载 doc string 模板和参数命名规范。
- 步骤 3 只加载文档组装模板和目录结构指南。
- 步骤 4 只加载质量检查 checklist 和术语表。

”因为它的价值不只是顺序清晰，还在于每一步只加载当前所需的 references 和 templates，能让 context 一直保持干净。”
——视频原话，05:57-06:07

图 4-2：Context 分段加载示意图		
步骤	加载的 References	Context 状态
解析 API	OpenAPI 规范、源码目录	精简
生成 doc string	文档模板、命名规范	精简
组装文档	目录结构指南、示例库	精简
质量检查	Checklist、术语表	精简

图 8: Pipeline 模式下，每一步只加载当前所需知识，避免 context 膨胀。来源：视频 06:00-06:07

常见误区：把 Pipeline 当成”多轮对话”

Pipeline 不是简单的”分多次问模型”，而是在 **skill 定义层就把流程固化下来**。如果只是把原来的长 prompt 拆成四句短 prompt 依次发送，而没有显式的门控和推进条件，模型仍然可能在某一轮”偷工减料”。真正的 Pipeline 要求：每一步的输出格式、校验规则、失败处理逻辑都在 `skill.md` 中写死，而不是依赖模型的”自觉”。

4.1.4 适用场景

Pipeline 模式特别适合以下类型的任务：

- **文档生成**：复杂文档需要分步构建，如技术白皮书、API 参考手册、发布说明。
- **复杂代码修改**：分析 → 修改 → 测试 → 审查的强依赖链条。
- **发布流程**：构建 → 测试 → 部署 → 验证，任何一步失败都必须回滚或阻塞。
- **合规流程**：审计、审批、签字等**不能跳过验证步骤**的任务。

”这类模式特别适合文档生成、复杂代码修改、发布流程、合规流程，这种不能跳过验证的任务。”
——视频原话，05:49–05:57

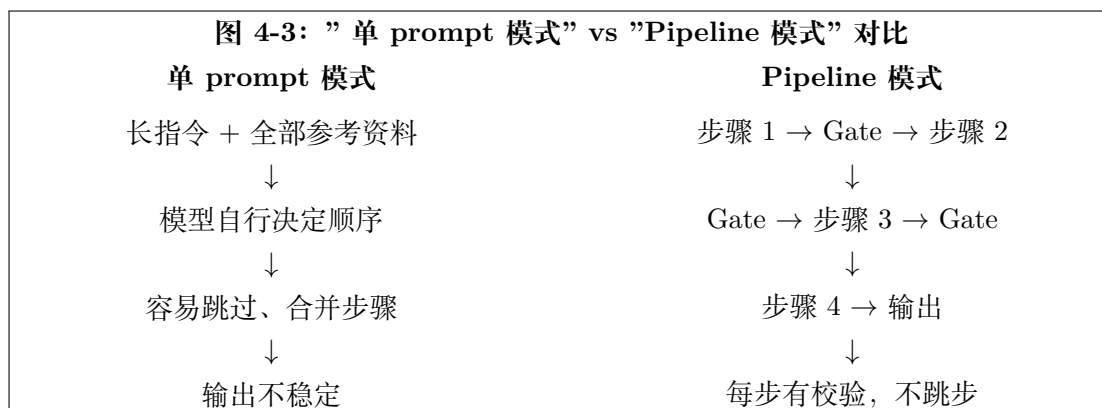


图 9: 单 prompt 模式依赖模型自律，Pipeline 模式用显式结构强约束。来源：视频 05:17–05:43

4.2 本章小结

Pipeline 模式是五种设计模式中唯一一个**把流程本身写进 skill**的模式。它解决的核心问题是：当任务天然具有强顺序依赖、且每一步都需要严格校验时，如何防止 Agent ”中途偷懒” 跳过关键步骤。

其核心设计要点可归纳为三点：

1. **工作流化**：把 `skill.md` 写成步骤清单，明确每一步的输入、输出和推进条件。
2. **显式门控**：在步骤之间设置 Gate，未通过校验禁止进入下一步。
3. **分段加载**：每一步只加载当前所需的 references 和 templates，保持 context 干净，减少干扰。

与 Inversion 模式（先问后做）和 Reviewer 模式（检查标准与流程分离）相比，Pipeline 更关注**执行阶段的步骤管控**。它常与 Generator 模式叠加使用——Pipeline 负责”按什么顺序生成”，Generator 负责”每一步按什么结构生成”。在下一章中，我们将把这五种模式放到一张决策树上，讨论如何根据实际问题选择合适的模式，以及如何将它们组合成更强大的 skill 架构。

5 模式选择与组合策略：如何选型、如何叠加

正如上一章末尾所提到的，五种模式逐一展开之后，接下来需要一张决策树来回答最后一个关键问题：面对一个真实的 skill 开发需求，到底该选哪一种？更进一步，它们之间能否协同使用？本章将围绕这两个问题，给出一张可直接落地的决策路径图，并讨论模式叠加的工程实践。

5.1 选择路径图：一张决策树搞定选型

Google 原文在最后给出了一段非常实用的判断路径。它不需要你背诵五种模式的完整定义，只需要你连续追问自己四个问题，就能快速定位最合适的模式。

这篇文章给了一个很实用的判断路径：先问这个 skill 是否要产出内容？如果要，而且是从模板里产出，那通常就是 Generator；要产出，但本质只是运行时加载某一套知识，那

更像 Trapper；如果这个 skill 是在评估已有输入，那通常是 Reviewer；如果它在开始之前必须先向用户补全信息，那就是 Inversion；如果它还有严格步骤、确认点、不能跳过的门，那就是 Pipeline。

将上述判断逻辑整理为一张决策树，如图 10 所示（视频时间：06:07-06:42）。

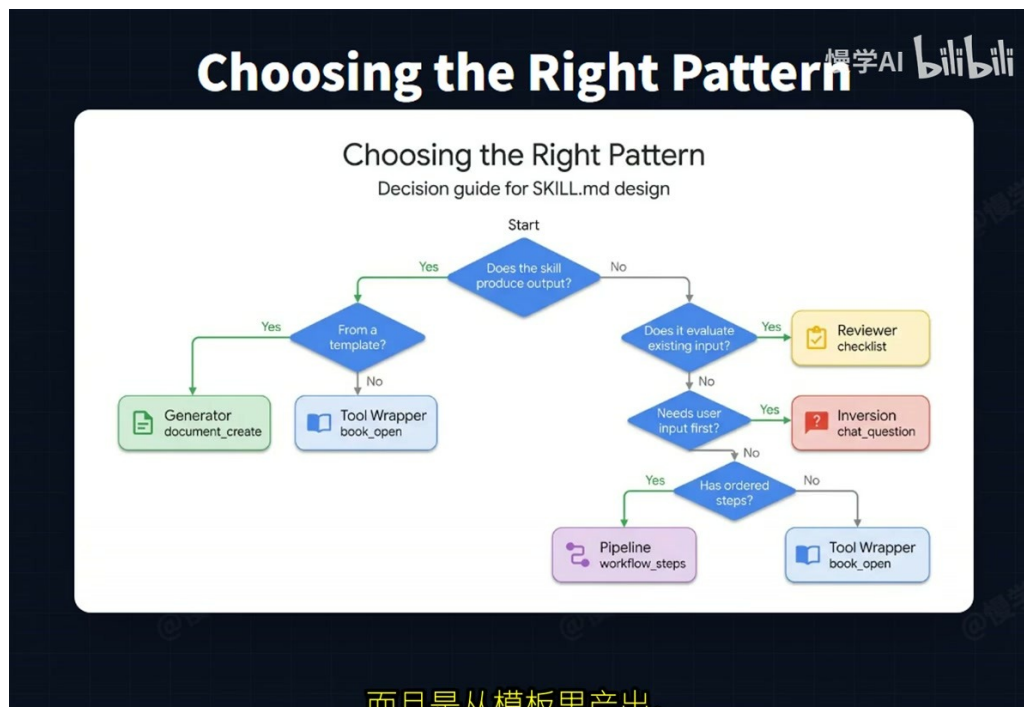


图 10: 模式选择决策树 (Decision Tree)，改编自 Google Cloud Tech 原文。来源：视频 06:07-06:42

决策路径的使用要点

这张决策树的关键在于区分 **Trapper** 与 **Generator** 的分支逻辑。两者都可能“产出”内容，但核心差异在于：Generator 依赖预定义的 **模板 (Template)** 来锁定输出结构；而 Trapper 的本质是 **按需注入知识 (On-demand Injection)**，运行时加载的是规范、约定或参考资料，而非固定模板。如果你的 skill 只是“在合适的时机把某套专家手册交给 Agent”，那它属于 Trapper；如果它还要进一步规定“产出的文档必须包含哪些字段、按什么顺序排列”，则属于 Generator。

建议读者在拿到一个新的 skill 需求时，先不要急着写 prompt，而是把这张决策树打印出来或贴在屏幕旁，逐条对照。多数情况下，四个问题问完，答案就已经非常清晰了。

5.2 模式叠加：1+1>2 的组合打法

五种模式并非互斥选项，而是可以互相叠加、协同工作的设计单元。理解这一点，是避免“为了选模式而选模式”的关键。

这些模式之间可以互相叠加。Generator 前面可以接 Inversion，结尾可以接 Reviewer；Trapper 几乎可以叠在任何模式上，负责补知识。

视频中的这段话（06:42-06:54）点明了模式叠加的核心思想。下面用一张分层示意图（图 11）展示几种常见的组合方式。

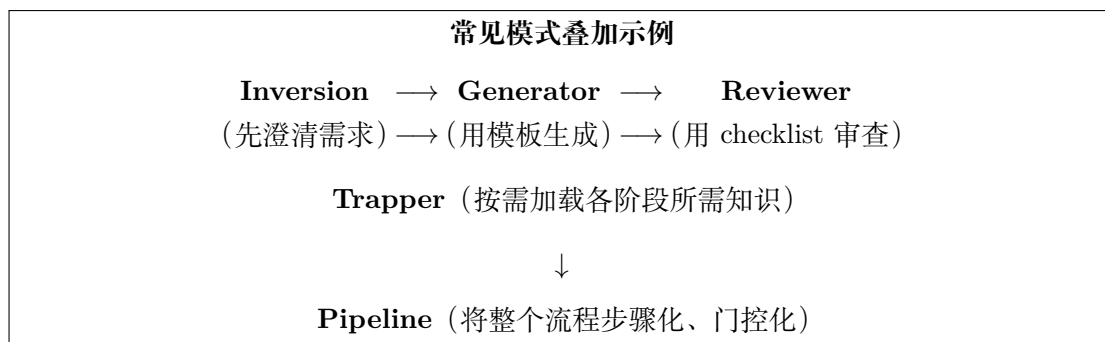


图 11: 模式叠加示例: Pipeline 为骨架, Inversion 为起点, Generator 为核心, Reviewer 为终点, Trapper 贯穿全程按需供知识。来源: 视频 06:42-06:54

模式叠加的工程类比

如果把构建一个 skill 比作做菜, 那么:

- **Pipeline** 是菜谱的步骤顺序——先备料、再下锅、最后装盘, 不能乱;
- **Inversion** 是备料确认——发现盐不够、火候条件不明, 先问清楚再动手;
- **Generator** 是标准化烹饪——用固定模具做蛋糕, 每次形状一致;
- **Reviewer** 是品控环节——出锅前对照 checklist 检查咸淡、色泽、温度;
- **Trapper** 是按需取食材——做川菜时取辣椒, 做粤菜时取蚝油, 不会把全厨房调料一次性倒在锅里。

叠加不是复杂化, 而是”各管一段、各司其职”。

以下是几组经过验证的常见组合:

- **Inversion → Generator**: 先用 Inversion 补齐需求信息, 再用 Generator 的模板生成标准化输出。适用于”需求模糊但必须产出规范文档”的场景, 例如客服机器人方案设计、API 文档初稿生成。
- **Generator → Reviewer**: 生成后立即进入审查流程, 用可替换的 checklist 做质量把关。适用于任何对输出质量有硬性要求的场景, 例如代码审查、合规报告校验。
- **Trapper + [任意模式]**: 在任意模式的执行过程中, 由 Trapper 按需加载该阶段所需的 references 和 templates。这是保持 context 干净、避免知识污染的最通用手段。
- **Pipeline 包裹 Inversion + Generator + Reviewer**: 将整个任务封装为一个强顺序工作流, 每一步都有明确的 gate。适用于复杂文档生成、发布流程、合规审批等”不能跳步”的场景。

叠加的常见误区

模式叠加虽然强大，但也要注意边界：

1. **不要为了叠加而叠加**。如果一个简单的 API 文档生成任务用 Generator 就能解决，没必要强行套一层 Pipeline。
2. **避免 gate 过度密集**。Pipeline 中的门控（Gate）应设置在“关键校验点”，而不是每一步都加，否则会导致交互冗长、用户体验下降。
3. **Trapper 不是万能缓存**。按需加载的前提是“能准确判断当前任务属于哪个领域”，如果判断逻辑本身写得很模糊，Trapper 反而会成为新的不稳定源。

5.3 总结与延伸

5.4 从“堆 prompt”到“选模式”

回顾全文，五种模式对应的是 Agent 开发中最常见的五类失控点：

失控点	模式	核心作用
知识什么时候加载	Trapper	按需注入，拒绝一次性塞满
输出结构不稳定	Generator	模板 + 风格指南锁定输出
审查标准与流程混为一谈	Reviewer	检查标准与检查流程硬拆开
信息不全却急着开工	Inversion	先问后做，信息补齐再生成
复杂任务跳步骤	Pipeline	把 workflow 写进 skill，每步有门控

表 2：五类失控点与五种模式的映射闭环

视频结尾有一段话，值得作为全文的核心观点反复咀嚼：

别再把越来越复杂的指令一股脑塞进同一个 system prompt。真正可靠的 skill，靠的是更合适的结构。你要先判断自己遇到的到底是知识问题、结构问题、审查问题、澄清问题还是流程问题，模式选对了，Agent 才会稳定。

这段话（06:54-07:14）揭示了一个深层认知转变：Agent skill 的可靠性不取决于 prompt 的长度或复杂度，而取决于**结构设计的合理性**。System prompt 膨胀（System Prompt Bloat）是一种常见的反模式——把所有逻辑、所有规范、所有约束都塞进同一个 prompt，结果往往是互相冲突、难以维护、输出漂移。

核心原则：结构化设计 > Prompt 堆砌

真正可靠的 skill 靠的是更合适的结构，而不是更长的指令。五种模式的价值，在于它们提供了一套经过验证的**结构化设计思维（Structured Design Thinking）**：遇到哪类问题，就引入哪类结构。这不是在增加复杂度，而是在用结构对抗模型的不确定性。

5.5 延伸思考：模式之外

在五种模式的基础上，还有三个值得进一步探索的方向：

1. **模式与工具链的映射**。Google ADK、Claude Code、LangChain 等框架对 skill.md 的语法支持各不相同，但五种模式的**设计思想**是跨框架通用的。下一步可以研究如何在不同框架中落地同一套模式。
2. **动态模式切换**。一个复杂的 Agent 可能在单次对话中先后经历”信息补全 (Inversion) → 知识加载 (Trapper) → 模板生成 (Generator) → 质量审查 (Reviewer)”的完整生命周期。如何在运行时自动识别当前阶段并切换模式，是一个值得深挖的工程问题。
3. **模式度量化评估**。如何衡量”引入某种模式后，Agent 输出的稳定性提升了多少”？可以建立一套评估指标，例如结构一致性得分、步骤完整率、审查漏检率等，用数据验证模式选择的有效性。

5.6 行动建议

如果你正在开发或维护一个 Agent skill，建议立即做以下三件事：

1. **拿起一张决策树**。对照图 10，判断你当前的 skill 主要对应哪一类问题。如果一个问题对应多个模式，优先考虑叠加而非取舍。
2. **检查你的 system prompt**。如果它已经超过 2000 tokens 且包含”如果 A 则做 B，如果 C 则做 D”式的复杂条件分支，那它很可能已经陷入了”堆 prompt”的反模式。尝试把其中的知识抽取到 references (Trapper)、把输出结构抽取到 template (Generator)、把审查规则抽取到 checklist (Reviewer)。
3. **从一个最小闭环开始**。不要试图一次性重构整个 skill。选择一个最痛的失控点，引入一种模式，跑通一个最小闭环，验证稳定性提升后，再逐步叠加其他模式。

模式选对了，结构搭好了，Agent 的输出自然稳定。这里是慢学 AI，我们下期再见。